

Introduction to Data Analysis

Week 1. Data Analysis Overview

Jin Hee Yoon

2026

Sejong University

Week 1 Overview

1주차에서는 **Data Analysis**가 무엇인지 이해하고, 실제 분석이 어떤 흐름으로 진행되는지 살펴본다. 또한 간단한 학생 성적 데이터를 이용하여 Python으로 데이터를 만들고, 요약하고, 시각화하는 기초 실습을 진행한다.

- **Week 1-1:** Data Analysis의 개념, 활용 분야, 질문 유형, 분석 절차
- **Week 1-2:** Python 실습 환경, DataFrame 생성, Basic Summary, Visualization, Correlation

이번 주의 핵심 목표는 데이터를 단순한 숫자의 모음이 아니라, 질문에 답하기 위한 근거로 이해하는 것이다.

Learning Objectives

이번 주 수업 후 다음 내용을 설명할 수 있어야 한다.

1. **Data Analysis**의 의미를 설명할 수 있다.
2. **Data-driven decision making**의 필요성을 이해할 수 있다.
3. **Descriptive, Diagnostic, Predictive, Prescriptive question**을 구분할 수 있다.
4. **Data Analysis Pipeline**의 각 단계를 설명할 수 있다.
5. Python에서 **pandas DataFrame**을 만들고 기본 요약을 확인할 수 있다.
6. 간단한 **Scatter plot**과 **Correlation**을 이용해 변수 관계를 탐색할 수 있다.

How to Study This Week

구분	주요 내용	학습 방식
Week 1-1	개념 중심	용어와 분석 흐름 이해
Week 1-2	실습 중심	Python 코드 실행 및 결과 해석

Python 코드는 눈으로만 보는 것보다 직접 실행하는 것이 중요하다. 같은 코드를 입력하고, 일부 값을 바꾸어 보면서 결과가 어떻게 달라지는지 확인한다.

Week 1-1. Concepts and Workflow

Week 1-1 Overview

Week 1-1에서는 Data Analysis의 기본 개념과 전체 흐름을 학습한다.

- Data Analysis란 무엇인가?
- 왜 Data Analysis를 배우는가?
- Data-driven decision making이란 무엇인가?
- Data Analysis에서 다루는 question의 유형
- Data Analysis Pipeline
- Data Science, Statistics, Machine Learning, AI의 관계
- 분석 결과를 해석할 때 주의할 점

Week 1-1의 목표는 실제 코딩 이전에 데이터분석의 목적과 사고방식을 이해하는 것이다.

What Is Data Analysis?

Data Analysis는 원자료인 **raw data**를 이해, 설명, 예측, 의사결정에 도움이 되는 정보로 바꾸는 과정이다.

일반적인 **Data Analysis**는 다음 활동을 포함한다.

- data를 수집하거나 불러온다.
- data의 구조와 변수의 의미를 이해한다.
- missing value, outlier, 오류를 점검한다.
- summary statistics와 visualization으로 pattern을 탐색한다.
- 필요한 경우 model을 만들고 결과를 해석한다.
- 분석 결과를 다른 사람에게 전달한다.

Data Analysis는 프로그램을 실행하는 기술만이 아니라, 질문을 데이터로 검토하고 근거 있는 결론을 만드는 과정이다.

Data Analysis as a Process

Data Analysis는 한 번에 끝나는 계산이 아니라, 여러 단계가 반복되는 과정이다.



실제 분석에서는 EDA 중 문제가 발견되면 다시 Cleaning으로 돌아가고, Modeling 결과가 부적절하면 다시 Question이나 Data 단계로 돌아갈 수 있다.

Raw Data and Information

Raw data는 아직 정리되거나 해석되지 않은 원래 상태의 데이터이다.

예를 들어 다음 값만 보면 의미를 알기 어렵다.

82, 60, 91, 73, 96

하지만 이것이 학생들의 **Final score**라는 것을 알면 평균, 최고점, 최저점, 성취 수준을 해석할 수 있다.

Data는 그 자체로 의미를 주지 않는다. 변수의 정의, 측정 방식, 분석 목적이 함께 주어질 때 정보가 된다.

Example: Same Numbers, Different Meanings

같은 숫자라도 어떤 변수인지에 따라 해석은 완전히 달라진다.

값	변수 의미	가능한 해석
82	Final score	비교적 높은 성적
82	Temperature	매우 높은 온도일 수 있음
82	Age	고령자 데이터일 수 있음
82	Attendance rate	출석률 82%

데이터분석에서 가장 먼저 해야 할 일은 값 자체를 보는 것이 아니라, 각 변수의 의미와 단위를 확인하는 것이다.

Why Do We Study Data Analysis?

Data Analysis는 다양한 분야에서 활용된다.

- **Business**: sales prediction, customer segmentation, demand forecasting
- **Engineering**: sensor monitoring, quality control, failure detection
- **Medicine**: patient risk analysis, clinical data summary
- **Education**: learning pattern analysis, student performance prediction
- **Public policy**: population, transportation, welfare data analysis
- **Artificial Intelligence**: machine learning 이전의 data preparation

좋은 분석 결과는 좋은 데이터 이해에서 시작된다. 데이터 구조, 변수의 의미, 결측치, 이상치를 먼저 확인해야 한다.

Data-driven Decision Making

Data-driven decision making은 경험이나 직관만으로 판단하지 않고, data에 근거하여 의사결정을 하는 방식이다.

예를 들어 한 카페에서 다음 질문을 생각할 수 있다.

- 어떤 시간대에 고객이 가장 많은가?
- 어떤 음료가 가장 많이 팔리는가?
- 비 오는 날에는 매출이 감소하는가?
- 할인 이벤트가 실제 매출 증가로 이어졌는가?

Data-driven decision making은 데이터를 통해 문제를 더 구체적으로 이해하고, 선택의 근거를 명확하게 만드는 과정이다.

Example: Decision Without Data vs. With Data

데이터 없이 판단	데이터 기반 판단
“아마 오후에 손님이 많을 것이다.”	시간대별 매출 데이터를 확인한다.
“이 메뉴가 인기 있을 것 같다.”	메뉴별 판매량과 재구매율을 확인한다.
“학생들이 어려워하는 것 같다.”	문항별 정답률과 과제 제출률을 확인한다.
“광고 효과가 있는 것 같다.”	광고 전후 매출과 방문자 수를 비교한다.

Data Analysis는 직관을 없애는 것이 아니라, 직관이 맞는지 데이터로 검토하는 과정이다.

Questions in Data Analysis

Data Analysis에서 좋은 **question**을 정의하는 것은 매우 중요하다.

분석 질문은 보통 다음과 같이 나눌 수 있다.

- **Descriptive question:** 무엇이 일어났는가?
- **Diagnostic question:** 왜 그런 일이 일어났는가?
- **Predictive question:** 앞으로 무엇이 일어날 것인가?
- **Prescriptive question:** 무엇을 해야 하는가?

분석 질문이 명확하지 않으면, 어떤 데이터를 모아야 하는지, 어떤 방법을 써야 하는지 결정하기 어렵다.

Descriptive Question

Descriptive question은 현재 또는 과거에 무엇이 일어났는지를 요약하는 질문이다.

예:

- 이번 학기 학생들의 평균 점수는 얼마인가?
- 지난달 매출은 얼마인가?
- 결석 학생은 몇 명인가?
- 제품별 판매량은 어떻게 다른가?

주로 사용하는 방법:

- count, mean, median, standard deviation
- frequency table
- simple visualization

Diagnostic Question

Diagnostic question은 어떤 현상이 왜 발생했는지 원인을 탐색하는 질문이다.

예:

- 왜 특정 반의 평균 점수가 낮았는가?
- 왜 특정 요일의 매출이 감소했는가?
- 왜 출석률이 낮은 학생의 성적이 낮은가?
- 왜 특정 제품에서 불량률이 높게 나타났는가?

Diagnostic question은 원인을 다루지만, 단순한 상관관계만으로 인과관계를 단정해서는 안 된다.

Predictive Question

Predictive question은 미래 값이나 아직 관측되지 않은 값을 예측하는 질문이다.

예:

- 다음 시험 점수를 예측할 수 있는가?
- 다음 달 매출은 어느 정도일까?
- 어떤 고객이 이탈할 가능성이 높은가?
- 환자가 고위험군에 속할 가능성은 얼마나 되는가?

주로 사용하는 방법:

- regression
- classification
- time series forecasting
- machine learning model

Prescriptive Question

Prescriptive question은 분석 결과를 바탕으로 어떤 행동을 해야 하는지 묻는 질문이다.

예:

- 어떤 학생에게 보충학습을 제공해야 하는가?
- 어느 시간대에 직원을 더 배치해야 하는가?
- 어떤 고객에게 할인 쿠폰을 보내야 하는가?
- 어떤 정책이 가장 효과적인가?

Prescriptive question은 단순한 분석 결과를 넘어 실제 의사결정과 연결된다.

Four Types of Questions: Student Example

학생 성적 데이터를 예로 들면 질문 유형은 다음과 같이 구분할 수 있다.

Question type	예시
Descriptive	Final score의 평균은 얼마인가?
Diagnostic	출석률이 낮은 학생의 성적이 왜 낮은가?
Predictive	StudyHours와 Midterm을 이용해 Final score를 예측할 수 있는가?
Prescriptive	어떤 학생에게 추가 학습 지원을 제공해야 하는가?

같은 dataset이라도 어떤 question을 설정하느냐에 따라 분석 방법이 달라진다.

Data Analysis Pipeline

일반적인 **Data Analysis Pipeline**은 다음과 같이 표현할 수 있다.

Question → Data → Cleaning → EDA → Modeling → Communication

- **Question:** 분석하고자 하는 문제 정의
- **Data:** 필요한 data 수집 또는 불러오기
- **Cleaning:** 결측치, 이상치, 오류 처리
- **EDA:** summary와 visualization으로 data 탐색
- **Modeling:** 예측 또는 설명 model 구축
- **Communication:** 결과 해석 및 전달

Step 1. Question

Question 단계에서는 분석의 목적을 명확히 정의한다.

좋은 분석 질문은 다음 조건을 갖는다.

- 너무 막연하지 않다.
- data로 확인할 수 있다.
- 변수와 분석 대상이 어느 정도 명확하다.
- 결과가 실제 해석이나 의사결정으로 연결된다.

나쁜 질문: “학생들이 공부를 잘하는가?”

좋은 질문: “StudyHours와 Attendance는 Final score와 어떤 관련이 있는가?”

Step 2. Data

Data 단계에서는 분석 질문에 필요한 데이터를 확보한다.

확인해야 할 사항:

- data source는 무엇인가?
- data는 언제 수집되었는가?
- 관측 단위는 무엇인가?
- 각 variable은 어떤 의미인가?
- sample size는 충분한가?
- 개인정보나 민감정보가 포함되어 있는가?

Data의 출처와 수집 방식을 모르면 분석 결과의 신뢰성을 판단하기 어렵다.

Step 3. Cleaning

Data Cleaning은 분석 전에 데이터의 문제를 확인하고 수정하는 단계이다.

주요 작업:

- **Missing value** 확인
- **Outlier** 확인
- 중복 row 확인
- 잘못된 data type 수정
- 변수 이름 정리
- 불가능한 값 확인

예: 시험 점수는 0점에서 100점 사이여야 한다. 만약 300점이 있다면 data entry error일 가능성이 높다.

Step 4. EDA

Exploratory Data Analysis (EDA)는 formal modeling 전에 data를 탐색하고 이해하는 과정이다.

대표적인 EDA 작업:

- row와 column 수 확인
- variable과 data type 점검
- descriptive statistics 계산
- distribution 확인
- relationship 시각화
- missing value와 outlier 탐지

EDA는 모델링보다 먼저 수행되어야 한다. 데이터의 문제를 모른 채 모델을 만들면 잘못된 결론을 얻을 수 있다.

Step 5. Modeling

Modeling은 data의 pattern을 수학적 또는 통계적 형태로 표현하는 단계이다.

예:

- StudyHours로 Final score를 예측하는 regression model
- 학생을 pass/fail로 분류하는 classification model
- 비슷한 고객을 묶는 clustering model
- 시간에 따른 매출을 예측하는 forecasting model

모델은 복잡할수록 항상 좋은 것은 아니다. 분석 목적에 맞고 해석 가능한 모델이 더 적절할 수 있다.

Step 6. Communication

Communication은 분석 결과를 이해하기 쉬운 형태로 전달하는 단계이다.

좋은 결과 보고에는 다음이 포함된다.

- 분석 질문
- 사용한 data의 설명
- 핵심 summary와 visualization
- 주요 결과
- 결과의 해석
- 한계점과 주의사항

Data Analysis는 숫자를 계산하는 것으로 끝나지 않는다. 결과가 어떤 의미인지 설명할 수 있어야 한다.

Data Science, Statistics, Machine Learning, AI

Data Analysis는 여러 분야와 연결된다.

용어	간단한 설명
Statistics	data의 변동성과 불확실성을 다루는 학문
Data Science	data 수집, 처리, 분석, 시각화, 모델링을 포함하는 넓은 분야
Machine Learning	data로부터 pattern을 학습하여 예측이나 분류를 수행하는 방법
AI	지능적 판단이나 행동을 수행하는 시스템을 만드는 분야

이 강의에서는 AI나 Machine Learning 이전에 반드시 필요한 Data Analysis의 기초를 먼저 다룬다.

Example Dataset: Student Scores

다음과 같은 학생 성적 **dataset**이 있다고 하자.

ID	StudyHours	Attendance	Assignment	Midterm	Final
1	3.5	90	85	78	82
2	1.0	65	70	55	60
3	4.5	95	90	88	91
4	2.0	75	80	70	73
5	5.0	100	95	92	96

가능한 analysis question:

- Final score의 평균은 얼마인가?
- StudyHours는 Final score와 관련이 있는가?
- 추가적인 학습 지원이 필요한 학생은 누구인가?

Observation and Variable

표 형식 data에서는 **observation**과 **variable**의 개념이 중요하다.

- **Observation**: 분석 대상 하나하나를 의미한다. 학생 데이터에서는 한 명의 학생이 하나의 observation이다.
- **Variable**: observation에 대해 측정된 특성이다. StudyHours, Attendance, Midterm, Final 등이 variable이다.
- **Value**: 특정 observation과 variable이 만나는 곳에 있는 값이다.

분석 단위가 학생인지, 수업인지, 시험인지에 따라 row의 의미가 달라진다. 따라서 observation의 정의가 매우 중요하다.

Numerical and Categorical Variables

학생 성적 dataset의 변수는 다음과 같이 구분할 수 있다.

Variable	Type	설명
ID	Identifier	학생을 구분하는 번호
StudyHours	Numerical	공부 시간
Attendance	Numerical	출석률
Assignment	Numerical	과제 점수
Midterm	Numerical	중간고사 점수
Final	Numerical	기말고사 점수

ID는 숫자로 저장되어 있어도 크고 작음을 비교하는 수치형 변수로 해석하지 않는다.

Analysis Result and Interpretation

Data Analysis는 숫자를 계산하는 것으로 끝나지 않는다.

예를 들어 Final score 평균이 80.4라고 하자.

- **Result:** $\bar{x} = 80.4$
- **Interpretation:** Final score 평균은 80.4이며, 전반적으로 중간 이상의 성취를 보인다.
- **Limitation:** dataset이 5명의 학생만 포함하므로 전체 학생에게 일반화하기 어렵다.

분석 결과를 보고할 때는 숫자, 해석, 한계점을 함께 제시해야 한다.

Common Mistakes in Data Analysis

초보자가 Data Analysis에서 자주 하는 실수는 다음과 같다.

- data의 의미를 확인하지 않고 바로 계산한다.
- missing value와 outlier를 확인하지 않는다.
- sample size가 매우 작은데 일반화한다.
- correlation을 causation으로 해석한다.
- 그래프의 축과 단위를 표시하지 않는다.
- 분석 방법은 복잡하지만 question이 명확하지 않다.

좋은 분석은 복잡한 방법에서 시작되는 것이 아니라, 명확한 질문과 정확한 데이터 이해에서 시작된다.

Week 1-1 Summary

Week 1-1에서 배운 내용을 정리하면 다음과 같다.

- Data Analysis는 raw data를 의사결정에 도움이 되는 정보로 바꾸는 과정이다.
- 분석 질문은 descriptive, diagnostic, predictive, prescriptive question으로 나눌 수 있다.
- Data Analysis Pipeline은 Question, Data, Cleaning, EDA, Modeling, Communication으로 구성된다.
- 분석 결과는 숫자뿐 아니라 해석과 한계점을 함께 제시해야 한다.

다음 영상에서는 Python을 이용하여 작은 dataset을 직접 만들고, 기본적인 summary와 visualization을 수행한다.

Week 1-2. Python Practice

Week 1-2 Overview

Week 1-2에서는 Python을 이용하여 Data Analysis의 가장 기본적인 실습을 수행한다.

- Python 실습 환경 소개
- pandas와 DataFrame
- 작은 학생 성적 dataset 만들기
- head(), info(), describe() 확인
- mean, max, min, sort 계산
- scatter plot 그리기
- correlation 계산하기
- 분석 결과를 문장으로 해석하기

이번 실습의 목표는 복잡한 코딩이 아니라, 데이터를 불러온 뒤 가장 먼저 무엇을 확인해야 하는지 익히는 것이다.

Python Practice Environment

Data Analysis 실습에는 다음 환경을 사용할 수 있다.

- **Google Colab**: 웹 브라우저에서 바로 Python 실행 가능
- **Jupyter Notebook**: 코드와 설명을 함께 작성하기 좋음
- **Anaconda**: Python과 주요 패키지를 한 번에 설치 가능
- **VS Code**: 코드 파일을 작성하고 실행하기 좋음

초반 실습에서는 Google Colab이나 Jupyter Notebook처럼 셀 단위로 코드를 실행할 수 있는 환경이 편리하다.

Important Python Libraries

이번 강의에서 자주 사용하는 Python library는 다음과 같다.

Library	주요 기능	대표 사용 예
pandas	table data 처리	DataFrame, read_csv
NumPy	numerical computation	array, mean, std
matplotlib	visualization	scatter, hist, plot
scikit-learn	machine learning	train-test split, model fitting

1주차에서는 pandas와 matplotlib을 중심으로 아주 기초적인 분석 흐름을 실습한다.

Python Example: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

- pandas는 표 형식 데이터를 다룰 때 사용한다.
- numpy는 수치 계산에 사용한다.
- matplotlib.pyplot은 그래프를 그릴 때 사용한다.

pd, np, plt는 자주 사용하는 library 이름을 짧게 부르는 관례이다.

What Is a DataFrame?

DataFrame은 행과 열로 구성된 table data를 저장하는 pandas의 핵심 자료구조이다.

- row는 보통 observation을 나타낸다.
- column은 variable을 나타낸다.
- 하나의 cell에는 하나의 value가 들어간다.

Observation	Variable 1	Variable 2	Variable 3
1	value	value	value
2	value	value	value
3	value	value	value

Python Example: Creating a Small Dataset

```
data = {  
    "ID": [1, 2, 3, 4, 5],  
    "StudyHours": [3.5, 1.0, 4.5, 2.0, 5.0],  
    "Attendance": [90, 65, 95, 75, 100],  
    "Assignment": [85, 70, 90, 80, 95],  
    "Midterm": [78, 55, 88, 70, 92],  
    "Final": [82, 60, 91, 73, 96]  
}  
  
df = pd.DataFrame(data)  
print(df)
```

여기서 df는 student score dataset을 저장한 DataFrame이다.

Dataset Description

방금 만든 dataset은 5명의 학생에 대한 학습 및 성적 정보를 포함한다.

Variable	Meaning	Type
ID	학생 번호	Identifier
StudyHours	하루 평균 공부 시간	Numerical
Attendance	출석률	Numerical
Assignment	과제 점수	Numerical
Midterm	중간고사 점수	Numerical
Final	기말고사 점수	Numerical

분석을 시작하기 전에 각 column이 무엇을 의미하는지 확인해야 한다.

Python Example: First Rows with head()

```
print(df.head())  
print(df.head(3))
```

head()는 DataFrame의 앞부분 row를 보여준다.

- data가 제대로 만들어졌는지 확인할 수 있다.
- column name이 올바른지 확인할 수 있다.
- 값의 형식이 예상과 맞는지 확인할 수 있다.

Python Example: Data Information with info()

```
print(df.info())
```

info()는 다음 정보를 보여준다.

- row 개수
- column 개수
- column별 non-null value 개수
- column별 data type
- memory usage

실제 data를 불러온 직후에는 head()와 info()를 먼저 확인하는 것이 좋다.

Python Example: Descriptive Statistics with describe()

```
print(df.describe())
```

describe()는 numerical variable의 기본 summary를 제공한다.

- count: 값의 개수
- mean: 평균
- std: 표준편차
- min, max: 최솟값과 최댓값
- 25%, 50%, 75%: quartile

Basic Summary: What Should We Check?

DataFrame을 만든 후에는 다음을 확인한다.

- row와 column은 몇 개인가?
- 어떤 variable이 있는가?
- numerical variable과 categorical variable은 무엇인가?
- missing value가 있는가?
- 값의 범위가 상식적으로 가능한가?
- 평균과 표준편차는 어느 정도인가?

Basic Summary는 단순한 출력이 아니라, 데이터가 분석 가능한 상태인지 확인하는 첫 점검 표이다.

Python Example: Selecting One Column

```
print(df["Final"])  
print(df["StudyHours"])
```

하나의 column을 선택할 때는 `df["column_name"]` 형식을 사용한다.

Column name은 정확히 입력해야 한다. 대문자와 소문자가 다르면 다른 이름으로 인식된다.

Python Example: Selecting Multiple Columns

```
score_cols = df[["Assignment", "Midterm", "Final"]]
print(score_cols)

learning_cols = df[["StudyHours", "Attendance", "Final"]]
print(learning_cols)
```

여러 column을 선택할 때는 column name의 list를 사용한다.

분석 목적에 따라 필요한 variable만 선택하면 data를 더 쉽게 이해할 수 있다.

Python Example: Mean, Min, Max

```
print(df["Final"].mean())  
print(df["Final"].min())  
print(df["Final"].max())  
  
print(df["StudyHours"].mean())  
print(df["Attendance"].mean())
```

이 코드는 Final score, StudyHours, Attendance의 기본 요약값을 계산한다.

평균은 데이터의 중심을 나타내지만, 이상치가 있으면 크게 영향을 받을 수 있다.

Interpreting Basic Summary

예를 들어 다음과 같은 결과가 나왔다고 하자.

Summary	Value
Mean of Final	80.4
Minimum of Final	60
Maximum of Final	96
Mean of StudyHours	3.2
Mean of Attendance	85.0

단순히 “평균은 80.4이다”라고 말하는 것보다, “기말고사 평균은 80.4점이며, 최저점은 60점, 최고점은 96점으로 학생 간 차이가 존재한다”라고 해석하는 것이 좋다.

Python Example: Finding the Best Student

```
best_index = df["Final"].idxmax()
best_student = df.loc[best_index]

print(best_index)
print(best_student)
```

- `idxmax()`는 가장 큰 값이 있는 row index를 반환한다.
- `loc[]`는 해당 index의 row를 선택한다.

이 방법을 사용하면 최고 점수뿐 아니라 그 학생의 다른 정보도 함께 확인할 수 있다.

Python Example: Filtering Rows

```
high_score = df[df["Final"] >= 80]
print(high_score)

low_attendance = df[df["Attendance"] < 80]
print(low_attendance)
```

Filtering은 조건을 만족하는 row만 선택하는 것이다.

예:

- Final score가 80 이상인 학생 선택
- Attendance가 80 미만인 학생 선택

Python Example: Multiple Conditions

```
selected = df[(df["StudyHours"] >= 3) & (df["Final"] >= 80)]  
print(selected)  
  
need_support = df[(df["Attendance"] < 80) | (df["Final"] < 70)]  
print(need_support)
```

조건이 여러 개일 때는 & 또는 |를 사용한다.

각 조건은 반드시 괄호로 묶어야 한다. 그렇지 않으면 오류가 발생할 수 있다.

Python Example: Sorting Data

```
print(df.sort_values("Final"))  
print(df.sort_values("Final", ascending=False))  
  
sorted_df = df.sort_values(["Attendance", "Final"], ascending=[False, False])  
print(sorted_df)
```

Sorting은 큰 값이나 작은 값을 빠르게 확인하는 데 도움이 된다.

성적순, 출석률순, 공부시간순으로 정렬하면 data의 구조를 직관적으로 볼 수 있다.

Visualization

Visualization은 data의 pattern을 그래프로 표현하는 과정이다.

Visualization을 사용하면 다음을 쉽게 확인할 수 있다.

- 값이 어디에 몰려 있는가?
- 변수 사이에 관계가 있는가?
- 이상치가 있는가?
- 그룹 간 차이가 있는가?
- 시간에 따른 변화가 있는가?

그래프는 복잡한 계산 전에 **data**의 전체적인 모양을 빠르게 이해하게 해준다.

Python Example: Scatter Plot

```
plt.scatter(df["StudyHours"], df["Final"])  
plt.xlabel("StudyHours")  
plt.ylabel("Final")  
plt.title("Study Hours and Final Score")  
plt.show()
```

Scatter plot은 두 numerical variable 사이의 관계를 확인할 때 사용한다.

점들이 오른쪽 위 방향으로 분포하면 두 변수 사이에 양의 관계가 있을 가능성이 있다.

Python Example: Midterm and Final Scatter Plot

```
plt.scatter(df["Midterm"], df["Final"])  
plt.xlabel("Midterm")  
plt.ylabel("Final")  
plt.title("Midterm and Final")  
plt.show()
```

이 그래프는 Midterm score가 높은 학생이 Final score도 높은 경향이 있는지 확인하는 데 사용된다.

Scatter plot은 correlation을 계산하기 전에 먼저 확인하면 좋은 그래프이다.

Python Example: Histogram

```
plt.hist(df["Final"], bins=5)
plt.xlabel("Final")
plt.ylabel("Frequency")
plt.title("Distribution of Final Scores")
plt.show()
```

Histogram은 하나의 numerical variable의 분포를 확인할 때 사용한다.

Final score가 어느 구간에 많이 분포하는지, 낮은 점수 학생이 있는지 확인할 수 있다.

Python Example: Bar Plot

```
score_names = ["Assignment", "Midterm", "Final"]
score_means = df[score_names].mean()

plt.bar(score_means.index, score_means.values)
plt.ylabel("Average Score")
plt.title("Average Scores by Assessment")
plt.show()
```

Bar plot은 범주별 값이나 요약값을 비교할 때 유용하다.

Correlation

Correlation은 두 numerical variable 사이의 linear relationship 강도를 나타낸다.

상관계수 r 은 다음 범위를 가진다.

$$-1 \leq r \leq 1$$

- $r > 0$: 한 변수가 증가할 때 다른 변수도 증가하는 경향
- $r < 0$: 한 변수가 증가할 때 다른 변수는 감소하는 경향
- $r \approx 0$: 뚜렷한 linear relationship이 약함

Correlation은 association을 나타낼 뿐, causation을 증명하지 않는다.

Python Example: Correlation

```
print(df["StudyHours"].corr(df["Final"]))  
print(df["Attendance"].corr(df["Final"]))  
print(df["Midterm"].corr(df["Final"]))
```

이 코드는 각각 다음 관계의 correlation을 계산한다.

- StudyHours와 Final
- Attendance와 Final
- Midterm과 Final

Python Example: Correlation Matrix

```
cols = ["StudyHours", "Attendance", "Assignment", "Midterm", "Final"]  
  
corr_matrix = df[cols].corr()  
print(corr_matrix)
```

Correlation matrix는 여러 numerical variable 사이의 correlation을 한 번에 보여준다.

Correlation matrix를 보면 어떤 variable들이 서로 비슷하게 움직이는지 빠르게 확인할 수 있다.

Interpreting Correlation Carefully

Correlation을 해석할 때는 주의가 필요하다.

예를 들어 StudyHours와 Final score의 correlation이 높게 나왔다고 하자.

올바른 해석:

- StudyHours가 많은 학생일수록 Final score가 높은 경향이 있다.

주의해야 할 해석:

- StudyHours가 Final score를 반드시 증가시킨다고 단정할 수는 없다.

Correlation이 causation을 의미하지는 않는다. 다른 요인, sample size, data collection 방식도 고려해야 한다.

Mini Case 1: Student Support

학생 성적 데이터를 이용해 추가 학습 지원 대상자를 찾는다고 하자.

가능한 기준:

- Final score가 70점 미만인 학생
- Attendance가 80% 미만인 학생
- StudyHours가 2시간 미만인 학생
- Midterm과 Final이 모두 낮은 학생

분석 기준은 항상 목적에 맞게 설정해야 한다. 단순히 점수 하나만으로 판단하기보다 여러 **variable**을 함께 보는 것이 좋다.

Python Example: Student Support Rule

```
df["NeedSupport"] = (df["Final"] < 70) | (df["Attendance"] < 80)

print(df[["ID", "Attendance", "Final", "NeedSupport"]])

support_students = df[df["NeedSupport"] == True]
print(support_students)
```

이 예제에서는 Final score가 낮거나 Attendance가 낮은 학생을 추가 지원 대상으로 표시한다.

Mini Case 2: Cafe Sales Data

Data Analysis는 학생 성적뿐 아니라 매출 데이터에도 적용할 수 있다.

카페 매출 데이터에서 가능한 question:

- 가장 매출이 높은 요일은 언제인가?
- Coffee와 Tea 중 어느 제품이 더 많이 팔리는가?
- 방문자 수가 많을수록 Revenue가 증가하는가?
- 비 오는 날에는 Revenue가 감소하는가?

같은 분석 방법이라도 dataset이 바뀌면 해석 문맥이 달라진다.

Python Example: Cafe Sales Dataset

```
sales = pd.DataFrame({  
    "Day": ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"],  
    "Visitors": [120, 95, 100, 110, 150, 230, 210],  
    "Coffee": [80, 65, 70, 75, 110, 180, 160],  
    "Tea": [30, 25, 28, 35, 40, 55, 60],  
    "Revenue": [480000, 390000, 410000, 455000, 650000, 1050000, 980000]  
})  
  
print(sales)  
print(sales.describe())
```

Python Example: Cafe Sales Visualization

```
plt.bar(sales["Day"], sales["Revenue"])
plt.xlabel("Day")
plt.ylabel("Revenue")
plt.title("Daily Revenue")
plt.show()

plt.scatter(sales["Visitors"], sales["Revenue"])
plt.xlabel("Visitors")
plt.ylabel("Revenue")
plt.title("Visitors and Revenue")
plt.show()
```

Bar plot은 요일별 Revenue 비교에 적합하고, Scatter plot은 Visitors와 Revenue의 관계를 확인하는 데 적합하다.

Mini Case 3: Health Check Data

건강검진 데이터에서도 Data Analysis를 사용할 수 있다.

가능한 question:

- 나이대별 평균 혈압은 어떻게 다른가?
- BMI가 높을수록 혈당이 높은 경향이 있는가?
- 특정 기준을 넘는 고위험군은 몇 명인가?
- 운동 시간이 긴 사람의 건강 지표가 더 좋은가?

건강 데이터는 민감정보를 포함할 수 있으므로 개인정보 보호와 윤리적 사용이 특히 중요하다.

Python Example: Health Check Dataset

```
health = pd.DataFrame({
    "ID": [1, 2, 3, 4, 5, 6],
    "Age": [22, 35, 47, 52, 29, 41],
    "BMI": [21.5, 25.2, 28.0, 30.1, 23.0, 27.4],
    "ExerciseHours": [5, 2, 1, 0, 4, 2],
    "BloodSugar": [88, 102, 115, 130, 95, 110]
})

print(health.head())
print(health.describe())
print(health["BMI"].corr(health["BloodSugar"]))
```

From Code to Interpretation

Python 코드를 실행한 뒤에는 반드시 결과를 문장으로 해석해야 한다.

예:

- 코드 결과: `df["Final"].mean() = 80.4`
- 해석: 이 dataset에서 학생들의 Final score 평균은 80.4점이다.
- 한계: 학생 수가 5명뿐이므로 전체 학생에 대한 결론으로 일반화하기 어렵다.

Data Analysis의 최종 결과는 코드가 아니라, 데이터를 근거로 한 설명이다.

Practice 1: Basic Summary

학생 성적 dataset을 이용하여 다음 질문에 답하라.

1. Final score의 mean을 계산하라.
2. Final score의 minimum과 maximum을 계산하라.
3. StudyHours의 mean을 계산하라.
4. Attendance가 가장 높은 학생을 찾으라.
5. Final score가 80점 이상인 학생만 선택하라.

각 결과에 대해 한 문장으로 해석을 작성해 보라.

Practice 1: Python Code

```
print(df["Final"].mean())
print(df["Final"].min())
print(df["Final"].max())
print(df["StudyHours"].mean())

best_attendance = df.loc[df["Attendance"].idxmax()]
print(best_attendance)

high_final = df[df["Final"] >= 80]
print(high_final)
```


Practice 2: Visualization

학생 성적 dataset을 이용하여 다음 그래프를 그려라.

1. Final score의 Histogram을 그려라.
2. StudyHours와 Final의 Scatter plot을 그려라.
3. Attendance와 Final의 Scatter plot을 그려라.
4. Assignment, Midterm, Final의 평균을 Bar plot으로 비교하라.

그래프를 그린 후에는 그래프에서 보이는 pattern을 한 문장 이상으로 설명해야 한다.

Practice 2: Python Code I

```
plt.hist(df["Final"], bins=5)
plt.xlabel("Final")
plt.ylabel("Frequency")
plt.title("Final Score Histogram")
plt.show()
```

```
plt.scatter(df["StudyHours"], df["Final"])
plt.xlabel("StudyHours")
plt.ylabel("Final")
plt.title("StudyHours and Final")
plt.show()
```

Practice 2: Python Code II

```
plt.scatter(df["Attendance"], df["Final"])
plt.xlabel("Attendance")
plt.ylabel("Final")
plt.title("Attendance and Final")
plt.show()

score_means = df[["Assignment", "Midterm", "Final"]].mean()
plt.bar(score_means.index, score_means.values)
plt.ylabel("Average Score")
plt.title("Average Score by Assessment")
plt.show()
```

Practice 3: Correlation

학생 성적 dataset을 이용하여 다음 correlation을 계산하라.

1. StudyHours와 Final의 correlation
2. Attendance와 Final의 correlation
3. Midterm과 Final의 correlation
4. Assignment, Midterm, Final 사이의 correlation matrix

Correlation 값이 크다고 해서 반드시 인과관계가 있다고 해석하면 안 된다.

Practice 3: Python Code

```
print(df["StudyHours"].corr(df["Final"]))
print(df["Attendance"].corr(df["Final"]))
print(df["Midterm"].corr(df["Final"]))

corr_matrix = df[["Assignment", "Midterm", "Final"]].corr()
print(corr_matrix)
```

Practice 4: Create Your Own Dataset

다음 조건을 만족하는 작은 dataset을 직접 만들어 보라.

- 최소 6개의 observation을 포함한다.
- 최소 4개의 variable을 포함한다.
- numerical variable을 2개 이상 포함한다.
- categorical variable을 1개 이상 포함한다.
- 분석 question을 2개 이상 작성한다.

예:

- 카페 매출 dataset
- 학생 학습 dataset
- 운동 시간과 건강 지표 dataset
- 온라인 쇼핑 구매 dataset

Practice 4: Example Code

```
mydata = pd.DataFrame({
    "Person": ["A", "B", "C", "D", "E", "F"],
    "Group": ["X", "X", "Y", "Y", "X", "Y"],
    "StudyHours": [2, 3, 1, 4, 5, 2],
    "SleepHours": [7, 6, 8, 5, 6, 7],
    "Score": [75, 82, 68, 90, 95, 78]
})

print(mydata)
print(mydata.describe())
print(mydata.groupby("Group")["Score"].mean())
```

Checklist for Week 1

1주차 학습 후 체크해 볼 사항

- Data Analysis의 정의를 설명할 수 있는가?
- Descriptive, diagnostic, predictive, prescriptive question을 구분할 수 있는가?
- Data Analysis Pipeline의 각 단계를 설명할 수 있는가?
- Python에서 DataFrame을 만들 수 있는가?
- `head()`, `info()`, `describe()`의 역할을 설명할 수 있는가?
- 기본 summary statistics를 계산할 수 있는가?
- Scatter plot과 correlation을 해석할 수 있는가?

Week 1 Final Summary

- Data Analysis는 raw data를 정보와 의사결정의 근거로 바꾸는 과정이다.
- 좋은 분석은 명확한 question에서 시작된다.
- EDA와 visualization은 data를 이해하는 데 필수적이다.
- pandas DataFrame은 table data를 다루는 기본 도구이다.
- 분석 결과는 숫자, 해석, 한계점을 함께 제시해야 한다.

다음 주에는 Data Types와 Data Structures를 더 자세히 다루고, Python의 기본 자료구조와 DataFrame 구조를 학습한다.

Review Exercise

1. 본인이 관심 있는 주제로 작은 dataset을 직접 생성하라.
2. 최소 5개의 observation과 4개의 variable을 포함하라.
3. `head()`, `info()`, `describe()` 결과를 출력하라.
4. numerical variable 2개를 선택하여 scatter plot을 그려라.
5. 두 numerical variable의 correlation을 계산하라.
6. 분석 결과를 5문장 이내로 해석하라.

코드만 제출하지 말고, 결과가 무엇을 의미하는지 간단히 설명해야 한다.

Review Exercise 1: Smartphone Usage I

Smartphone Usage dataset을 생성하자.

```
# Create a smartphone usage dataset
phone = pd.DataFrame({
    "Student": ["A", "B", "C", "D", "E", "F", "G", "H"],
    "ScreenTime": [2.0, 5.5, 3.0, 7.0,
                  4.0, 6.5, 2.5, 5.0],
    "SleepHours": [8.0, 5.5, 7.5, 5.0,
                  7.0, 5.5, 8.0, 6.0],
    "StudyHours": [4.0, 1.5, 3.5, 1.0,
                  3.0, 1.5, 4.5, 2.0] })

print(phone)
```

이 dataset은 8명의 학생에 대한 ScreenTime, SleepHours, StudyHours 정보를 포함한다.

Review Exercise 1: Smartphone Usage II

Dataset의 기본 정보를 확인한다.

```
print(phone.head())  
  
phone.info()  
  
print(phone.describe())
```

- head()로 앞부분의 row를 확인한다.
- info()로 column과 data type을 확인한다.
- describe()로 numerical variable의 summary를 확인한다.

Review Exercise 1: Smartphone Usage III

ScreenTime과 SleepHours의 관계를 분석한다.

```
# Draw a scatter plot
plt.scatter(phone["ScreenTime"], phone["SleepHours"])
plt.xlabel("Screen Time")
plt.ylabel("Sleep Hours")
plt.title("Screen Time and Sleep Hours")
plt.show()

# Calculate correlation
corr_value = phone["ScreenTime"].corr(
    phone["SleepHours"]
)

print(corr_value)
```

Review Exercise 1: Interpretation

분석 결과를 해석한다.

1. ScreenTime이 긴 학생일수록 SleepHours가 짧아지는 경향이 나타난다.
2. Scatter plot에서 두 variable 사이의 negative relationship을 확인할 수 있다.
3. Correlation 값이 음수이면 두 variable이 서로 반대 방향으로 변화하는 경향이 있음을 의미한다.
4. 그러나 ScreenTime이 증가하여 SleepHours가 감소했다고 인과관계를 단정할 수는 없다.
5. Observation 수가 8개뿐이므로 결과를 전체 학생에게 일반화하기에는 한계가 있다.

Review Exercise 2: Temperature and Electricity Usage I

Temperature와 ElectricityUsage를 포함하는 dataset을 생성하자.

```
# Create an electricity usage dataset
energy = pd.DataFrame({
    "Day": ["Mon", "Tue", "Wed", "Thu",
           "Fri", "Sat", "Sun"],
    "Temperature": [24, 26, 28, 30, 32, 34, 35],
    "ElectricityUsage": [210, 225, 250, 280,
                        315, 355, 380],
    "Humidity": [55, 58, 60, 62, 65, 67, 70]
})

print(energy)
```

각 row는 하루를 나타내며, Temperature와 ElectricityUsage는 numerical variable이다.

Review Exercise 2: Temperature and Electricity Usage II

Dataset의 기본 정보를 확인한다.

```
print(energy.head())  
  
energy.info()  
  
print(energy.describe())
```

- Observation과 variable의 수를 확인한다.
- 각 variable의 data type을 확인한다.
- Temperature와 ElectricityUsage의 범위와 평균을 확인한다.

Review Exercise 2: Temperature and Electricity Usage III

Temperature와 ElectricityUsage의 관계를 분석한다.

```
# Draw a scatter plot
plt.scatter( energy["Temperature"], energy["ElectricityUsage"] )

plt.xlabel("Temperature")
plt.ylabel("Electricity Usage")
plt.title("Temperature and Electricity Usage")
plt.show()

# Calculate correlation
corr_value = energy["Temperature"].corr(energy["ElectricityUsage"])

print(corr_value)
```

Review Exercise 2: Interpretation

분석 결과를 해석한다.

1. Temperature가 높은 날일수록 ElectricityUsage가 증가하는 경향이 나타난다.
2. Scatter plot에서 두 variable 사이의 positive relationship을 확인할 수 있다.
3. Positive correlation은 Temperature와 ElectricityUsage가 같은 방향으로 변화하는 경향이 있음을 의미한다.
4. 그러나 Temperature만으로 ElectricityUsage의 변화를 모두 설명할 수는 없다.
5. Observation 수가 적고 다른 요인을 고려하지 않았으므로 결과 해석에 주의해야 한다.

본 교육콘텐츠는 교육부와 한국연구재단의 지원을 받아 제작한 AI융합
인재양성사업단의 콘텐츠입니다.